

Project #02, part 02: EB deployment (v1.0)

Assignment: Deploy Photoapp web service to AWS EB

Submission: via Gradescope

Policy: team-oriented, late work is accepted

Complete By: Monday May 11th @ 11:59pm CST

Late submissions: see syllabus for late policy... No submissions accepted after Thurs 05/14 @ 11:59pm

Pre-requisites: you must complete Project 02 web service, and your score on "Project 02 – web service" should be 60/60 to receive full credit

Overview

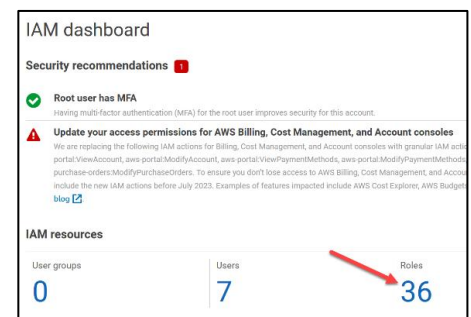
Congratulations, you have built a non-trivial web service using JavaScript, Node.js, and Express! Well done. The programming is over, the last part is to configure AWS to run your web service using **EC2**, Amazon's *Elastic Compute* service. Since we are using a well-known framework, AWS makes it even easier by providing a service on top of EC2 called *Elastic Beanstalk* (EB) that makes setup even easier. Recall that we created scripts in Lab 03 to deploy a web service using EB; we'll extend those scripts to deploy your Photoapp web service to AWS EB, enabling world-wild access.

(1) AWS IAM Roles

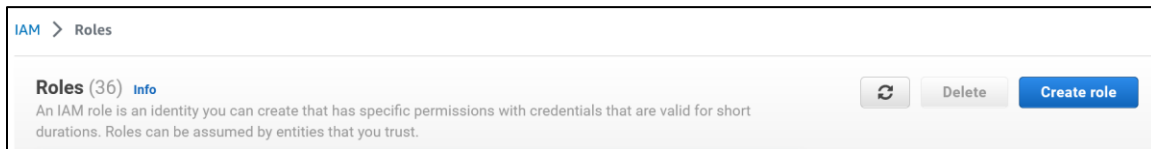
[NOTE: some students have reported these security roles already exist, so this step may be optional.]

The first step is to setup two security roles in AWS IAM (identity access management), which are needed to provide EB and EC2 with access to your AWS infrastructure. Steps:

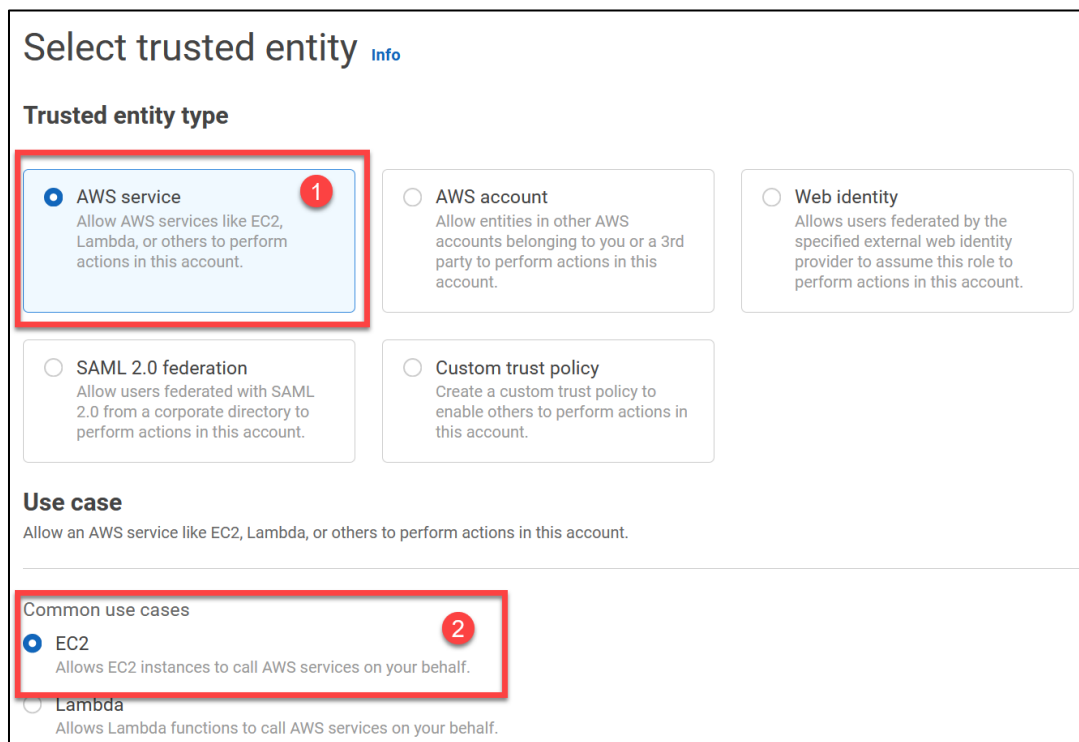
1. Login to AWS, open the Management Console. Search for "IAM".
2. You'll see 1 or more roles --- click on the number (36 in our case):



3. Click “Create role” to create a new role:



4. On the next screen, select “AWS service” and “EC2” as common use case:



<< continued on next page >>

5. The next screen is for adding additional permissions --- narrow down the search by entering "AWSElastic" in the search field and pressing ENTER. Then select the two roles shown and click "Next":

Add permissions [Info](#)

Permissions policies (Selected 2/848) [Info](#)

Choose one or more policies to attach to your new role.

Filter policies by property or policy name and press enter.

"AWSElastic" X

Clear filters

-

Policy name

☐	AWSElasticBeanstalkWebTier
☐	AWSElasticBeanstalkWorkerTier
☐	AWSElasticBeanstalkMulticontainerDocker
☒	AWSElasticBeanstalkEnhancedHealth
☐	AWSElasticBeanstalkCustomPlatformforEC2Role
☐	AWSElasticBeanstalkRoleWorkerTier
☐	AWSElasticBeanstalkRoleSNS
☐	AWSElasticBeanstalkRoleRDS
☐	AWSElasticBeanstalkRoleECS
☐	AWSElasticBeanstalkRoleCore
☐	AWSElasticBeanstalkRoleCWL
☐	AWSElasticBeanstalkReadOnly
☐	AdministratorAccess-AWSElasticBeanstalk
☒	AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy
☐	AWSElasticDisasterRecoveryRecoveryInstancePolicy

Cancel

Previous

Next

6. On the next screen name the role “**aws-elasticbeanstalk-service-role**” and click “Create Role”:

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and '+=,._@-' characters.

[Cancel](#)[Previous](#)[Create role](#)

7. After a few minutes the role should be created --- if anything goes wrong, you can delete and start over.
8. We need to repeat this process and create another role named “**aws-elasticbeanstalk-ec2-role**”. Repeat steps 2 – 6, except when you “Add permissions”, select these 3 policies instead:

IAM > Roles > aws-elasticbeanstalk-ec2-role

aws-elasticbeanstalk-ec2-role

Summary

Creation date

April 05, 2023, 19:54 (UTC-05:00)

Last activity

✓ 26 minutes ago

[Permissions](#)[Trust relationships](#)[Tags](#)[Access Advisor](#)[Revoke](#)

Permissions policies (3) [Info](#)

You can attach up to 10 managed policies.

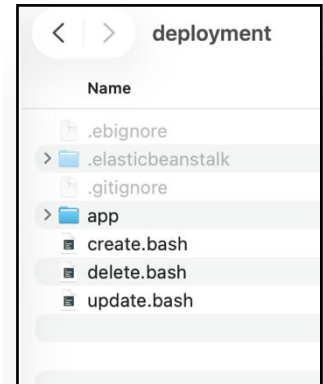
Filter policies by property or policy name and press enter.

	Policy name ↗	Type
☐	☐ AWSElasticBeanstalkWebTier	AWS manag...
☐	☐ AWSElasticBeanstalkWorkerTier	AWS manag...
☐	☐ AWSElasticBeanstalkMulticontainerDocker	AWS manag...

(2) AWS Elastic Beanstalk

Now that the roles are created, we can use the scripts from Lab 03 to automate the deployment of your Photoapp web service to EB. Steps:

1. Find the scripts you created from Lab 03. Here's the [handout](#) if you didn't complete Lab 03, or if you want a quick review of Lab 03 and the 3 scripts: create, update, and delete. Recall there were 2 versions: .bash scripts for Linux and Mac users, and .ps1 for Windows users.
2. Make a copy of the entire folder containing your scripts. In the following steps, work with the copy you just made...
3. Open the "app" sub-folder, and delete the contents.
4. Copy your web service files into the "app" sub-folder: all the .js files, and the "photoapp-config.ini" file.
5. Download a copy of "package.json" from [dropbox](#) and save into "app" sub-folder; this file is used by EB to determine what additional Node.js modules are needed by the web service.
6. Open the **create** script (either "create.bash" or "create.ps1") in a text editor. Change the variables that define the application name and environment name: these currently define a calculator web service, change to something related to photoapp. Update the other scripts to match the new names.
7. Next, the **create** script needs to be modified so when the web service is created, it specifies the IAM roles we created earlier: **aws-elasticbeanstalk-service-role**, and **aws-elasticbeanstalk-ec2-role**. Here's how to make those modifications...
8. Open a Terminal / PowerShell window, and run: **eb create --help**
9. The **help** option shows you the command-line options for the "eb create" command. Find the command-line argument for specifying the **service role**. Back in your create script file, scroll down to the "eb create" command, and add the command-line option to specify the service role as **aws-elasticbeanstalk-service-role**. [**NOTE:** the "eb create" command spans multiple lines, where each line ends with \ in a .bash script and ` in a .ps1 script.]
10. Use the help system again to identify the command-line argument for specifying the **instance profile**, and then modify the eb create command in your create script, adding the command-line option to specify the instance profile as **aws-elasticbeanstalk-ec2-role**. [**NOTE:** pay close attention to the command-line option: one uses "_" while the other uses "-".]
11. Finally, we want to configure the web service to use **basic** health monitoring, disabling AWS checks that possibly stop our web service during normal throughput testing (i.e. when we purposely overload the web service with calls to see how quickly it responds to increasing numbers of client requests). Your create script current has 4 steps. After the 4th step, add the following (this is the text for a .bash script, for a .ps1 script replace the \ with `):



```
echo ""
echo "5. Configuring for basic health monitoring..."
aws elasticbeanstalk update-environment \
  --environment-name $ENV_NAME \
  --option-settings Namespace=aws:elasticbeanstalk:healthreporting:system,OptionName=SystemType,Value=basic
```

12. Save the changes to your create script, and then run via Terminal / PowerShell window. Wait for the script to complete. Assuming success, use "eb status" to obtain the endpoint, which is CNAME (see screenshot below):

```
hummel> eb status
Environment details for: calc-web-service-env
Application name: calc-web-service
Region: us-east-2
Deployed Version: app.zip
Environment ID: e-ks3ujqppxg
Platform: arn:aws:elasticbeanstalk:us-east-2::platform/Node.js 24 running on 64bit Amazon Linux 2023/6.7.3
Tier: WebServer-Standard-1.0
CNAME: calc-web-service-env.eba-fvqeytij.us-east-2.elasticbeanstalk.com
Updated: 2026-02-06 19:16:01.304000+00:00
Status: Ready
Health: Green
```

13. The endpoint **URL** for your web service becomes **http://CNAME**
14. At this point your web service is now running in AWS via EC2. A quick test is to open a web browser and browse to **URL/users** or **URL/images** --- if all is well, you should get data back from your web service! You can even try **URL/image/1001** and download an image from S3.
15. Recall your client API functions in "photoapp.py" lookup the URL in the config file "photoapp-client-config.ini". This file has the format:

```
[client]
webservice=http://localhost:8080
```

Notice it's "http" and not "https", with NO / at the end of the URL. Find the folder for your client-side code, and modify the "photoapp-client-config.ini" file to have the endpoint URL for your web service. When you're done, this should look something like this:

```
[client]
webservice=http://your_environment_name...us-east-2.elasticbeanstalk.com
```

Now run your client / tests with your new config file --- your web service should work exactly as before. Your web service is now running world-wide! [*NOTE: due to EB limitations in our current configuration, images larger than 1MB will fail to upload successfully.*]

Grading and Electronic Submission

Make sure your web service is up and running in EB; if you aren't sure, test by browsing to your web service and executing **/users**. Here in part 02 you only need to submit your two configuration files. If you were running the docker image to test your client-side code, you can submit as follows:

```
/gradescope/gs submit 1288073 8052817 photoapp-config.ini photoapp-client-config.ini
```

Otherwise you can drag-drop these two config files to the Gradescope site "Project 02 - EB". The goal is an autograder score of 10/10; you have unlimited submissions. At this point you have completed project 02, and

if all went well, your combined score from all the Gradescope submissions should be 100/100. You've built a real, multi-tier app running in the cloud – congratulations and well done!

Pausing the database, deleting the web service...

Go ahead and pause your database, and delete your web service via the delete script since we won't be using this web service in future projects. It's also easy to redeploy via the create script if you want to demo it or extend in the final project.